장애 대응 Agent 개선 결과보고서

광주 3 반 G085 신동운

선택 과제: 과제 1, Base AI Agent 기능 개선 및 추가

프로젝트 개요

제공된 Base AI Agent 를 바탕으로 장애 질문에 대한 조사 대상 확인과 대응 문서 검색, 답변의 근거 검증을 개선했다. 사용자가 서비스 상태를 질문하면 관측값을 조회하고 관련 런북을 검색한 뒤 사실, 가설, 조치와 한계를 구분해 보여 주는 서비스다. 런북은 장애 대응 절차를 담은 문서다.

사용자가 답변의 근거를 직접 확인할 수 있도록 구성했다. 대상이 모호하면 먼저 확인하고, 일부 조회가 실패하면 성공한 결과를 남긴다. 조치 문장은 문서의 원문 단계에서 가져와 적용 조건이 답변 작성 과정에서 섞이지 않도록 했다.

주요 결과

GPT-4o mini 로 최종 v5 와 원본을 동일한 30 문항에서 세 차례 비교했다. 매회 응답 반환은 원본 27/30, v5 30/30 이었다. 필수 문서 17 문항의 재현율 평균은 원본 70.6~76.5%, v5 100%였다. 기존 기능과 결함 수정 테스트 48 개가 통과했고, 실제 모델을 사용한 브라우저 검증도 완료했다. Ollama 모델도 별도로 측정해 모델 선택 근거를 보완했다.

실습용 모니터링 데이터를 바탕으로 장애 조사와 대응 절차 안내까지 구현했다. 비교 실험에는 개발 과정에서 사용한 30 문항을 적용했다.

문제 배경과 개선 목표

장애 대응에서 중요한 것은 그럴듯한 설명보다 조사 대상과 근거를 정확히 연결하는 일이다. 예를 들어 "로그인이 느리다"는 질문만으로 서비스를 정하면 다른 대상을 조사할 수 있다. 두 서비스를 함께 조회하다 하나가 실패했을 때는 전체 분석을 중단하지 않고 확인된 결과와 확인하지 못한 부분을 나눠야 한다.

대응 문서의 조건도 유지해야 한다. "일정 시간 안에 회복되지 않으면 수행하는 조치"와 "정상화 이후 수행하는 조치"를 하나로 요약하면 실행 시점이 달라질 수 있다. 이번 프로젝트는 이러한 문제를 줄이기 위해 대상 확인, 관측 기반 검색, 원문 조치 연결과 검증을 중심으로 기존 서비스를 개선했다.

개발 및 검증 범위

개발 대상은 제공된 all_total_vue_student 코드이며, 최종 구현 버전은 v5 다. 기존 모니터링 도구와 화면을 활용하고 검색, 보고서 생성, 오류 처리 흐름을 수정했다. 비교 실험에서는 원본 런북 6 개를 유지해 문서 추가의 영향과 코드 개선의 영향을 구분했다. 검증은 동일 입력 30 문항 비교, 자동 테스트 48 개, 실모델 브라우저 확인으로 나누어 진행했다.

1. 진행 사항과 7 개 개선 기능

원본은 모니터링과 MCP 도구 연결을 갖추고 있었지만, 대상이 없는 질문에서 기본 서비스를 선택하거나 단어가 비슷한 런북을 고를 수 있었다. 답변의 형식을 검사하는 것만으로는 관측 수치와 출처, 조치 조건의 일치를 확인하기 어려웠다. 또한 한 대상의 오류가 전체 조사 결과에 영향을 줄 수 있었다.

영역	무엇을 바꾸나	구현 내용과 확인 근거
1. RAG 검색	키워드 중심 선택을 관측 기반 의미 검색으로 개선	알림에 적용 가능한 문서를 추린 뒤 임베딩으로 순위를 계산한다. 정상 상태와 문서 없음은 no_match 로 처리한다. 검색 성공 17 문항 평가.
2. 근거 보고서	자유 형식 답변을 구조화	사실, 가설, 조치, 한계, 출처를 분리한다. 사실은 관측값으로 구성하고 조치는 원문 단계와 연결한다. 30 문항 사실 131 개 대조.
3. 검증과 재작성	형식 검사를 근거 검사로 확장	수치, 대상, 출처 ID 와 원문 조치를 대조한다. 최초 작성 포함 최대 3 회이며 미통과 초안은 공개하지 않는다. 상한과 오류 피드백 테스트.
4. 처리 화면	진행과 근거를 함께 표시	기존 SSE 전송을 활용해 조사 단계, 대상별 결과, 검증 상태와 런북 원문을 확인하도록 화면을 개선했다. POST/SSE 일치와 UI 검증.
5. 전후 평가	동일 조건의 비교 근거 추가	질문, 관측, 문서, 모델, 제한 시간을 고정하고 버전별 기록을 남긴다. 원본과 v5 직접 비교 및 v4 대비 회귀 확인.
6. 대상 확인	임의 대상 선택 제거	모호한 질문은 조회 전에 대상 선택을 요청한다. 보류 질문을 저장하고 선택 후 원 질문으로 재개한다. 선택 전 도구 호출 0 회 확인.
7. 부분 실패	작업별 실패 격리	기존 재시도에 대상별 실패 격리를 더했다. 성공한 결과를 유지하고 실패한 대상은 상태와 사유를 따로 표시한다. 부분 실패와 전체 실패 테스트.

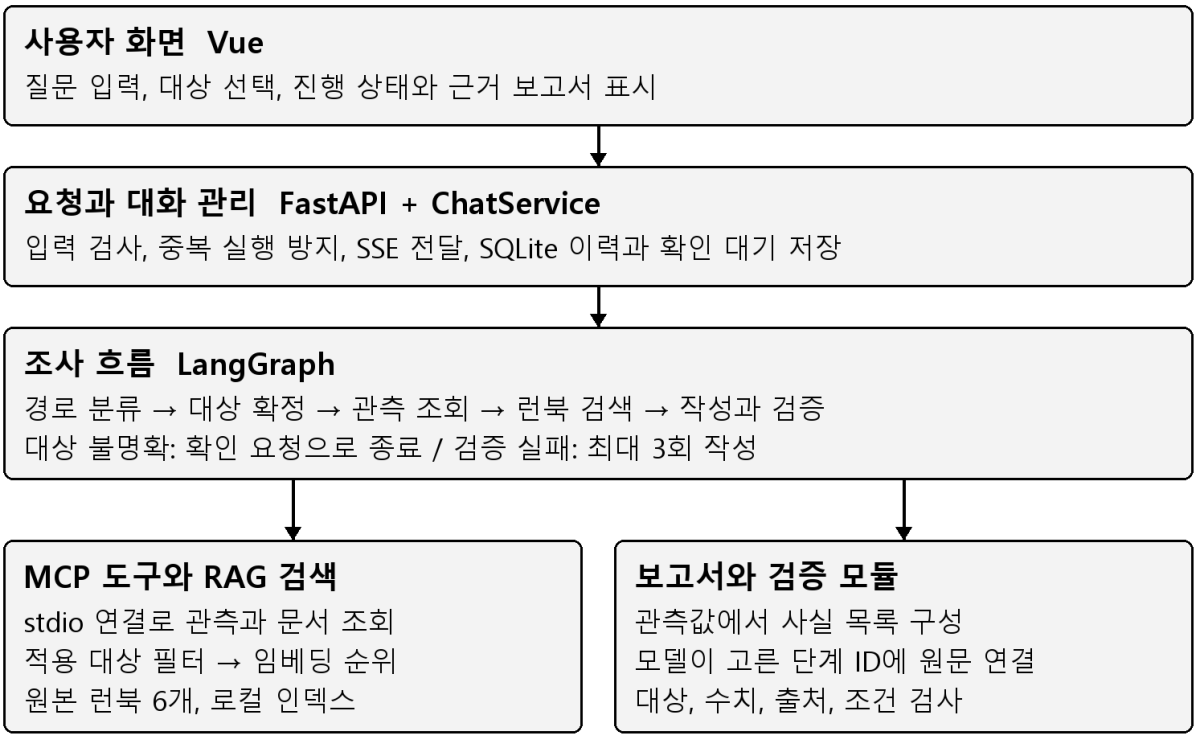
구현 범위

최종 v5 에는 위 일곱 기능을 구현했다. 검색 대상은 원본 런북 6 개로 유지했다. 문서 증가와 코드 개선의 효과를 섞지 않기 위해서다. 별도 분석 노드 대신 하나의 보고서에서 대상별 근거를 분리했고, 공통 원인을 자동 확정하지 않는다.

구현 범위는 기존 LangGraph 조사 흐름의 개선이다. 검색과 검증 등 주요 역할을 모듈로 분리했으며, 전문 서브에이전트, 사람의 승인 절차, 대화 요약은 이번 범위에서 제외했다. 벡터 검색은 로컬 인덱스를 사용한다.

2. 사용한 기술과 처리 구조

전체 흐름은 질문을 조사 가능한 대상으로 바꾸고, 관측과 문서를 연결한 뒤 답변을 검사하는 순서다. 모니터링 도구는 기존 MCP 연결을 활용하고, 검색 정책과 보고서 검증을 추가했다.



조회 실패: 성공한 대상의 결과를 유지하고 실패 사유를 표시

검증 미통과: 초안의 가설과 조치를 제거하고 확보한 사실만 안내

실제 모델 API 사용 / 모니터링 관측은 실습용 데이터

원본에서 활용한 구조	이번에 개선한 처리
FastAPI, Vue, LangGraph, MCP stdio, SSE, 의존성 주입, 모델 어댑터, SQLite, 모의 관측	대상 확인, 관측 기반 RAG, 근거 보고서, 원문 조치 연결, 검증과 재작성, 부분 실패 처리, 비교 평가

2.1. 검색과 검증의 구현 방식

질문이 답변이 되는 과정

Vue 화면에서 질문을 보내면 FastAPI 가 입력을 검사하고 ChatService 가 대화별 실행과 중복 요청을 관리한다. LangGraph 는 일반 대화와 장애 질문을 구분한다. 장애 질문의 대상이 모호하면 확인 요청으로 종료하며, 대상 선택 응답을 받은 뒤 원 질문을 이어서 처리한다.

대상 확정 → MCP 관측 조회 → 적용 가능한 런북 검색 → 근거 보고서 작성 → 검증 → 결과 표시

조회 결과는 대상별 성공 또는 실패 상태로 저장한다. 성공한 관측으로 문서를 검색한다. 검색 결과가 없고 활성 알림이 있으면 해당 알림으로 검색어를 바꿔 보완한다. 작성 결과가 검증에 실패하면 오류를 전달해 다시 작성한다. 상한까지 실패하면 초안 대신 확보한 사실과 제한 사항을 안내한다. 일반 대화의 토큰 스트리밍은 제공하지 않으며 진행 이벤트와 최종 답변을 구분한다.

RAG 검색

retrieval/index.py 는 text-embedding-3-small 의 512 차원 벡터를 사용한다. 짧은 런북의 조건과 순서를 보존하려고 문서 전체를 한 청크로 저장하고, incident_type 과 문서 해시를 메타데이터에 포함한다. 인덱스와 원문 해시가 다르면 불일치로 처리한다.

먼저 applicability.py 가 실제 알림에 맞는 문서만 후보로 남긴다. 후보의 정규화 벡터와 관측 알림, 사용자 질문의 코사인 유사도를 계산한다. 관측 0.8, 질문 0.2 의 가중 점수를 사용하며, 0.31 이상이면서 최고 점수와 차이가 0.03 이내인 문서를 최대 3 개 반환한다. 이 값은 실험에서 고정했다.

사실과 조치 조건의 보존

agent/reporting.py 는 수집한 관측으로 사실 목록을 구성한다. 모델은 가설에 연결할 근거와 조치 단계 ID 를 선택하고, action_catalog.py 가 해당 단계의 원문을 가져온다. validation.py 는 수치, 대상, 출처 연결과 조치 문장을 대조한다.

실패 처리와 상태 관리

nodes/investigation.py 는 시도당 모니터링 15 초, 런북 검색 45 초 제한을 적용하며 최초 호출을 포함해 최대 3 회 시도한다. 시간 초과와 연결 오류는 재시도하지만 영구 오류는 실패로 기록한다. service.py 는 동일 요청의 SSE 재연결에 결과를 재사용한다. SQLite 에는 대화 이력과 대상 확인 대기를 저장하며 전체 그래프 체크포인트나 완료 요청 캐시를 재시작 후 복원하지는 않는다.

원본 재사용과 직접 개선한 부분

FastAPI, Vue, LangGraph, MCP stdio 연결, 모델 어댑터, SQLite 와 모의 데이터는 원본을 활용했다. 개선한 부분은 검색 정책, 대상 확인, 보고서와 검증, 조건 보존, 부분 실패 대응, 평가 체계다. 원본의 의존성 주입 구조를 활용하고 실행 제한 설정과 테스트 대역을 확장했다. LangSmith 외부 추적은 비활성화했다.

3. 비교 실험의 설계

기존 30 문항은 단일 장애뿐 아니라 표현이 바뀐 질문, 복합 대상, 근거 부족과 도구 실패를 포함한다. 원본과 최종 v5 모두 같은 질문, 관측값과 오류 조건을 사용했다. 검색 문서를 6 개로 고정해 데이터 증가에 따른 효과를 제외했다.

문항 유형	개수	확인할 동작
단일 대상 장애	6	대상과 관측값에 맞는 런북 검색
유사 표현	3	CPU, 메모리, 디스크를 간접 표현한 질문의 처리
복합 대상	6	대상별 문서와 사실 분리, 정상 대상 구분
정상 상태와 문서 없음	3	정상 대상을 장애로 단정하지 않고 근거 없는 조치 생략
대상 확인	5	대상이 없거나 미등록 대상이 섞이면 조회 전에 확인
도구 실패	4	부분 실패, 전체 시간 초과, 검색 실패, 일시 오류 재시도
일반 대화와 기억	3	일반 대화 분류, 이전 이름 기억

자동 채점 기준

경로와 조사 대상은 사전에 정한 정답과 정확히 일치하는지 검사한다. 런북 검색은 문항별 필수 문서 ID 를 기준으로 한다. 검색 성공률은 필수 문서 중 하나 이상을 찾은 문항의 비율이고, 재현율은 문항별로 찾은 필수 문서 수를 전체 필수 문서 수로 나눈 값의 평균이다. 런북이 필요한 17 문항에 적용한다.

문서 없음은 불필요한 런북을 반환하지 않았는지, 대상 확인은 확인 질문을 반환하고 관측 조회를 실행하지 않았는지 검사한다. 응답 반환에는 대상 확인과 정상적인 실패 안내도 포함하므로, 검증 실패와 실행 오류는 별도 집계한다. 제한 시간 안에 결과를 저장하지 못한 문항은 정답 지표를 실패로 계산했다.

답변 내용 검토

관측 사실 일치, 출처 연결, 사실과 가설의 구분, 조치 조건, 근거 부족 시 대응을 확인했다. 수치와 원문은 원시 관측 및 런북과 자동 대조하고, 원인 단정과 실패 안내의 표현은 답변 내용을 읽어 검토했다. 이 내용 검토에는 AI 도구를 사용했으며 외부 평가자의 독립 채점은 수행하지 않았다.

반복 측정 조건

10 월 20 일의 원본과 v5 측정에 더해 10 월 21 일에 원본→v5 순서로 두 회차를 추가했다. 같은 회차의 두 버전을 직접 비교하며 추가 실행은 순차 진행했다. 질문과 실행 조건은 고정하고 생성 모델은 실제 API 를 사용했다. 테스트 데이터와 고정 도구 응답을 사용한 그래프 평가이며, 브라우저와 실제 MCP 연결 검증은 4 장에서 따로 제시한다.

3.1. 동일 조건의 원본과 최종본 비교

첫 회차의 비교 결과

첫 회차에서는 원본과 v5 를 같은 30 문항으로 비교했다. 분류와 작성 모델은 gpt-4o-mini-2024-07-18, temperature 0, 문항당 제한 240 초, 반복 1 회다. 질문, 관측, 오류 주입, 런북 6 개와 주요 실행 패키지를 동일하게 맞췄다. 실제 모델 API 를 호출하되 도구 경계에는 고정 데이터를 주입했다. 실제 MCP 연결과 브라우저 검증은 별도로 수행했다. 측정일은 2026 년 10 월 20 일이다.

검색 성공률은 필요한 문서를 하나 이상 찾은 문항의 비율이다. 재현율 평균은 각 문항의 필수 문서 중 찾은 비율을 구한 뒤 평균한다. 두 검색 지표의 분모는 런북이 필요한 17 문항이다. 대상 일치의 분모는 확인 질문 등을 제외한 25 문항이다. 대상 확인 지표는 조회를 실행하지 않고 확인 질문을 반환했는지를 자동으로 판정한다.

지표와 분모	원본	최종 v5	차이
응답 반환 30 문항	27/30	30/30	+10.0%p
경로 분류 일치 30 문항	96.7%	100%	+3.3%p
대상 정확 일치 25 문항	92.0%	100%	+8.0%p
필수 문서 검색 성공 17 문항	88.2%	100%	+11.8%p
필수 문서 재현율 평균 17 문항	76.5%	100%	+23.5%p
문서 없음 처리 3 문항	0/3	3/3	3 건 증가
대상 확인 요청 5 문항	0/5	5/5	5 건 증가

응답 반환과 실패의 해석

v5 의 30 건은 완료 20 건, 부분 결과 4 건, 대상 확인 5 건, 전체 조회 실패 안내 1 건이다. 원본은 주입된 실패 사례에서 ValueError 2 건과 문항 실행 중 TimeoutError 1 건이 발생했고, v5 는 이를 결과와 한계 안내로 반환했다. v5 의 연결 오류, 문항 실행 시간 초과, 최종 검증 실패는 각각 0 건이다. 내부 도구 실패를 처리한 부분 결과와 실행 자체의 오류를 구분했다.

자동 대조에서 관측 사실 131 개와 조치 원문 49 개가 일치했고 검사한 출처 링크 오류는 0 건이었다. v4 대비 동일 자동 지표의 하락은 없었다.

3.2. 반복 측정과 사용량

첫 회차 결과가 우연히 좋았던 것인지 확인하기 위해 같은 30 문항을 두 번 더 실행했다. 각 회차는 원본과 최종 v5 의 같은 질문을 짝지어 비교했다. 표의 지연시간은 두 버전에서 모두 응답한 질문만 골라 계산한 중앙값이다.

회차	응답 반환	문서 재현율 평균	공통 응답 중앙값	문항 수
1 차 10 월 20 일	27/30 → 30/30	76.5% → 100.0%	10.91 초 → 8.63 초	27
2 차 10 월 21 일	27/30 → 30/30	70.6% → 100.0%	6.52 초 → 6.48 초	27
3 차 10 월 21 일	27/30 → 30/30	76.5% → 100.0%	6.04 초 → 6.94 초	27

v5 는 매회 완료 20 건, 부분 결과 4 건, 대상 확인 5 건, 전체 조회 실패 안내 1 건으로 종료했다. 실행 오류와 최종 검증 실패는 없었다. 세 회차의 관측 사실 393 개와 선택된 조치 147 개는 원시 관측 및 원문과 일치했고, 검사한 출처 연결 오류는 0 건이었다. 선택된 조치 수는 회차별로 49 개, 45 개, 53 개였다.

생성 토큰과 추가 호출

30 문항 실행의 API 응답 사용량을 합산했다. 생성 토큰은 입력과 출력의 합이며 실패한 문항에서 기록된 사용량도 포함한다. v5 검색 임베딩 토큰은 별도로 적었다. 원본에는 이 호출이 없고, 인덱스를 미리 만드는 비용은 온라인 처리 집계에서 제외했다.

회차	원본 생성 토큰	v5 생성 토큰	v5 임베딩 토큰	재작성 전→후
1 차 10 월 20 일	26,967	63,585	936	0 → 1
2 차 10 월 21 일	27,142	59,012	936	0 → 0
3 차 10 월 21 일	26,909	59,289	936	0 → 0

생성 토큰은 원본의 2.17~2.36 배였다. 주로 입력 토큰이 늘었으며, 근거와 조치 목록을 전달하는 프롬프트와 추가로 반환한 답변이 집계에 반영됐다. 검색과 검증을 강화한 대신 사용량이 증가했다. 실제 청구 금액은 집계하지 않았다.

결과의 적용 범위

세 회차는 같은 개발용 30 문항을 반복한 결과다. 새 장애에 대한 일반화 성능이나 여러 사용자 부하를 평가한 것은 아니다. 날짜별 서비스 부하와 응답 변동이 있어 지연시간은 각 회차의 관측값으로 제시한다. 개선의 주된 근거는 검색, 대상 확인과 오류 처리 결과다.

3.3. 같은 질문에 대한 답변 변화

아래는 첫 회차에서 같은 질문에 대한 원본과 v5 의 실제 응답을 발췌하거나 요약한 내용이다.
예외로 종료된 실행에는 답변이 없다고 표시했다.

조사 대상이 없는 질문

질문: "로그인 응답이 느려요, 원인 확인해주세요"

원본	최종 v5
auth-service 를 선택해 CPU 78.0%, 메모리 61.0%, 커넥션 풀 92%를 제시하고 조치를 안내했다.	조사 전에 대상 확인을 요청했다. 이 단계의 모니터링 호출은 0 회였다. 화면에서 선택한 뒤 원 질문을 이어서 처리한다.

변화: 로그인이라는 표현만으로 조사 대상을 확정하던 동작을 없앴다. 선택 후 재개는 별도 브라우저 검증에서 확인했다.

해당 문서가 없는 장애

질문: "web-01 배포 서명 검증 실패 장애를 진단해줘"

원본	최종 v5
질문에 없는 auth-service 의 커넥션 풀 92%를 포함하고, 풀 크기 확대와 타임아웃 설정을 안내했다.	web-01 의 서명 검증 실패 알림만 표시했다. "관련 런북 없음; 추가 근거 수집이 필요합니다."라고 안내하고 구체 조치를 생략했다.

변화: 다른 서비스에서 실제로 관측된 수치라도 이번 질문의 근거가 될 수 없다는 점을 대상 검사와 문서 없음 처리에 반영했다.

한 대상의 조회가 실패한 질문

질문: "payments-service 와 cache-service 의 장애 원인을 확인해줘"
조건: cache-service 조회에 영구 오류를 주입했다.

원본	최종 v5
ValueError 로 종료되어 최종 답변을 반환하지 못했다.	payments-service 의 CPU 91.0%와 메모리 55.0%를 남기고 런북을 연결했다. cache-service 는 조회 실패로 상태와 원인을 판단할 수 없다고 안내했다.

변화: 조사 전체를 중단하지 않고 확보한 결과와 확인하지 못한 대상을 분리했다. 두 대상의 공통 원인은 확정하지 않았다.

3.4. 최종 모델 선택을 위한 비교

Ollama 의 실행 버전과 GPU 초기화 문제가 확인돼 과거 측정과 분리해 다시 비교했다. 원본과 v5 모두 Ollama 0.40.6 의 CPU 서버를 사용했다. CPU 는 Intel Core i7-1260P, 메모리는 약 32GB, 기본 컨텍스트 길이는 4096 이다. 질문, 관측, 모델 파일, 240 초 제한과 온도 0 을 맞췄다. 표의 구성은 분류 / 작성 모델 순서다.

모델 구성	응답 반환 전→후	문서 재현율 전→후	v5 검증 실패	240 초 초과 전→후
Qwen3 0.6B / 0.6B	28/30 → 21/30	73.5% → 47.1%	8	0 → 9
Qwen3 0.6B / 8B	25/30 → 23/30	64.7% → 58.8%	0	2 → 7

240 초 제한 초과 외의 실행 오류는 0.6B 단독 원본 재측정: TimeoutError 1 건, ValueError 1 건; 0.6B 단독 v5: 없음; 0.6B / 8B 원본 재측정: ValueError 2 건, TimeoutError 1 건; 0.6B / 8B v5: 없음. 원본의 주입된 도구 오류와 v5 가 정상적으로 반환한 실패 안내를 구분했다.

검증 실패 안내도 응답 반환 건수에 포함된다. 로컬 모델은 순차 실행했지만 PC 의 다른 프로세스 부하까지 고정하지는 않았다. 과거 실행 기록은 부록 G 에 별도로 정리했다.

제출 기본 모델을 GPT-4o mini 로 정한 이유

GPT-4o mini 는 세 회차 모두 30 문항을 반환하고 최종 검증 실패가 없었다. Qwen3 0.6B 단독 구성은 21/30 문항을 반환했으며 검증 실패가 8 건이었다. 0.6B / 8B 구성은 23/30 문항을 반환했고 검증 실패가 0 건이었다. 제출본의 기본 생성 모델은 반복 검증과 실제 화면 검증을 완료한 GPT-4o mini 로 유지했다.

0.6B 단독 구성의 v5 는 문서 없음 3 문항과 대상 확인 5 문항을 올바르게 처리했지만, 조치가 포함된 장애 보고서는 완성하지 못했다. 검증 실패에는 존재하지 않는 사실 ID 와 조치 단계 ID 선택이 포함됐다. 보고서 작성용 모델과 제어 흐름의 동작을 구분해서 판단했다.

8B 작성 모델 구성에서 반환된 보고서의 관측 사실 67 개와 조치 44 개는 원시 관측 및 원문과 일치했다. 다만 7 문항이 제한 시간을 넘겼으므로 전체 질문에 안정적으로 대응했다고 보기는 어렵다.

로컬 실행의 범위와 대가

Ollama 는 생성 모델을 로컬에서 실행하지만, 이번 RAG 임베딩은 text-embedding-3-small API 를 사용한다. 따라서 현재 구성은 완전한 오프라인 서비스가 아니다. 운영비를 더 줄이려면 로컬 임베딩으로 바꾸고 문서 검색 기준을 다시 검증해야 한다.

원본에서 반환된 답변과 개선 후 검증을 통과한 답변은 검사 수준이 다르다. 모델 선택에서는 응답 반환 수뿐 아니라 검증 실패, 근거가 부족할 때의 안내와 제한 시간 안에 끝나는지도 함께 살폈다.

4. 실제 화면에서 확인한 기능 동작

최종 v5 를 별도 포트 8107 에서 실행하고, 부분 실패 전용 8108 에서 cache-service 모니터링에만 오류를 주입했다. 생성 모델은 실제 GPT-4o mini, MCP 는 실제 stdio 연결이다. 관측값은 실습용 모의 데이터이며 응답 생성에는 테스트 대역 대신 실제 모델 API 를 사용했다. 대상 선택 재개와 두 보충 질문을 포함한 8 개 요청을 저장했다.

시나리오	결과	확인한 동작
정상 질문	완료	payments-service 의 관측과 출처 일치
복합 장애	완료	두 대상의 근거 분리, 공통 원인 확정 유보
대상 확인	확인 후 완료	선택 전 조회 0 회, 선택 후 auth-service 만 조사
부분 실패	부분 결과	payments 결과 유지, cache 판단 유보
조치 조건 첫 질문	완료	2 단계만 선택, 90% 적용 조건 보존
전체 절차 보충	완료	1~5 단계의 원문과 서로 다른 조건 보존
정상 상태 보충	완료	healthy, 알림 없음, 불필요한 구체 조치 없음

보고서 7 개는 앱 검증을 통과했고 대상 선택 전 1 개는 검증 미실행이 정상이다. 원시 MCP 출력과 대조한 관측 사실 39/39, 조치 원문 11/11 이 일치했다. API 응답 20 회의 모델명과 사용량을 기록했지만 자식 프로세스의 임베딩 사용량은 이 집계에 포함하지 않았다. 30 문항 결과와 이번 UI 수치는 서로 다른 실험의 결과이므로 구분해 해석해야 한다.

화면에서 확인한 대상 선택 흐름

"로그인 응답이 느려요"라는 질문에는 조사 대상 선택 화면이 표시됐다. 이 단계에서는 관측 도구를 호출하지 않았다. auth-service 를 선택한 뒤에는 원 질문을 유지하면서 해당 서비스만 조회했고, 관측 사실과 근거가 연결된 보고서를 반환했다. 확대 화면은 부록 F 에 실었다.

내용 검토에서는 사실과 가설의 구분, 실패 대상에 대한 판단 유보, 공통 원인 단정 여부를 확인했다. 내용 검토에는 AI 도구를 활용했다.

5. 조치 조건 보존과 트러블슈팅

미회복 시 조치와 정상화 후 조치를 분리

개발 과정에서 답변을 요약하면서 '5 분 내 미회복 시 트래픽 분산'과 '정상화 후 로그 분석'이 섞이는 문제가 있었다. 조치 문장을 자유롭게 다시 쓰게 하는 대신, 모델이 단계 ID 를 선택하고 서버가 원문을 연결하도록 바꿨다. 원문의 4 단계에는 5 분과 p95 1.0 초 미회복 조건이, 5 단계에는 정상화 후 조건이 남는다.

이번 첫 질문에서는 모델이 2 단계만 선택했다. 이를 전체 조건 검증으로 간주하지 않고 첫 결과를 보존했다. 별도 질문에서 전체 1~5 단계를 요청했을 때 모든 단계와 조건의 일치를 확인했다. 부록 F 의 전체 절차 화면은 이 보충 질문의 결과다.

전체 절차 응답에서는 미회복 시 조치와 정상화 후 조치가 원문 단계별로 구분되어 표시됐다. 세부 문장을 읽을 수 있도록 해당 화면을 부록 F 의 마지막 페이지에 확대해 실었다.

v4 에서 발견한 실행 결함과 v5 수정

문제	수정	재검증
날짜를 서비스로 오인	유효한 YYYY-MM-DD 만 대상 이름에서 제외	날짜 관련 5 개 테스트, 실모델 날짜 질문
일반 대화가 이벤트 루프 차단	동기 HTTP 호출을 to_thread 로 실행	비동기 호출 2 개 테스트
시작 실패 시 MCP 와 DB 잔류	종료 콜백 선등록, 실패와 취소 시 역순 정리	초기화와 MCP 7 개 테스트

기존 34 개와 신규 14 개, 총 48 개 자동 테스트가 통과했다. 모델 대역을 사용한 자동 검사이며 실제 MCP 시작, 호출, 종료 검사는 별도로 수행했다. 수정 전후의 오류 재현 결과도 보관했다. 코드는 오류 재현과 테스트 결과를 함께 검토했다.

Ollama 초기화 오류를 답변 실패와 분리

추가 측정에서 Ollama 가 답변을 생성하기 전에 HTTP 500 을 반환했다. 서버 로그의 Vulkan 연산 초기화 실패를 확인하고, 기존 서버를 유지한 채 11435 포트에 CPU 전용 서버를 실행했다. 원본과 v5 모두 같은 서버와 모델 파일로 재측정했다.

오류가 발생한 첫 실행도 별도로 보관했다. 비교용 실행에서는 프로그램 코드를 바꾸지 않고 평가 도구의 접속 주소만 변경했다. CPU 측정 결과를 과거 실행 환경의 속도와 직접 비교하지 않았다.

6. 한계와 트레이드오프 및 추가 개선

검색 범위를 제한한 대가

관측 알림에 적용 가능한 문서만 남기는 정책은 무관한 런북을 줄이지만, 새로운 알림 표현이나 알려지지 않은 장애에서는 관련 문서를 놓칠 수 있다. 문서 6 개와 알려진 표현을 중심으로 검증했으므로 대규모 운영 문서에서의 효과는 확인하지 않았다. 다음 단계에서는 코드 수정에 사용하지 않을 질문을 미리 확정하고 문서와 표현 범위를 넓혀 평가할 필요가 있다.

조치 단계 누락 검사

원문을 연결하면 문장이 길어지고 사용자가 조건을 직접 읽어야 한다. 실제 UI 첫 질문에서는 2 단계만 선택됐고, 전체 절차를 요청한 보충 질문에서 1~5 단계가 표시됐다. 전체 절차 요청에 대한 단계 누락 검사와, 선행 단계가 필요한 조치의 의존성 검증을 추가 개선 대상으로 둔다.

검증이 늘린 지연과 비용

임베딩 검색과 보고서 작성, 실패 시 재작성은 외부 API 호출과 지연을 늘릴 수 있다. 작성은 최대 3 회로 제한했다. 이번에는 반복 실험의 생성 토큰, 검색 임베딩 토큰과 재작성 횟수를 분리해 기록했다. 실제 청구 금액과 동시 사용자 부하까지는 측정하지 않았다.

실행과 사용자 경험의 남은 제한

to_thread 는 이벤트 루프 차단을 줄이지만 취소된 동기 HTTP 작업 자체를 즉시 중단하지는 않는다. MCP 공유 연결의 잠금 때문에 직렬화되는 구간도 있어 다중 사용자 부하 검증이 필요하다. 날짜 처리는 YYYY-MM-DD 형식에 맞췄으며 자연어 날짜와 서비스 별칭은 추가 처리가 필요하다.

대상 선택 후 이전 '대상 확인 필요' 결과가 이력 아래에 남아 최신 결과와 혼동할 수 있다. 진행 이벤트도 노드 완료 시 전달되므로 개별 병렬 작업의 즉시 갱신과 다르다. 후속 개선에서는 이전 확인 요청의 완료 상태 표시와 진행 상태의 세분화를 우선할 수 있다.

평가 범위와 운영 적용

GPT-4o mini 는 같은 30 문항을 세 회차로 비교하고, Ollama 는 별도 구성으로 측정했다. 반복 측정에도 개발에 사용한 질문이라는 제약은 남는다. 새로운 질문과 더 많은 문서, 별도 평가자의 내용 검토가 다음 평가에서 필요하다.

본 서비스는 로컬 실습용이다. 운영 모니터링 연동, 로그인과 권한 관리, 실제 조치 실행, 사람의 조치 승인과 운영 장애 복구는 구현하지 않았다. MCP 연결은 제공된 코드의 구조를 활용했다.

개발 결과

이번 개선에서는 조사 대상을 확인하고, 관측에 맞는 문서를 찾고, 사실과 조치의 근거를 추적할 수 있는 흐름을 구현했다. 제출 기본 모델인 GPT-4o mini 의 동일 입력 비교와 실제 UI 검증에서 검색, 대상 확인과 오류 처리의 개선을 확인했다. 이후 개선은 기능 수를 늘리기보다 단계 선택의 완전성, 새로운 질문 평가, 재현 가능한 설치와 실행에 우선순위를 둔다.

6.1. 개발을 통해 확인한 점과 이후 개선

답변을 만드는 것보다 근거를 유지하는 일이 중요했다

이번 작업에서는 답변의 형식을 맞추는 것만으로 충분하지 않았다. 보고서가 완성돼도 다른 대상의 수치가 섞이거나, 조치를 요약하면서 적용 시점이 바뀔 수 있었다. 이 때문에 관측 사실은 서버에서 구성하고, 조치는 모델이 문장을 다시 쓰는 대신 런북 단계의 원문과 연결하도록 바꿨다.

전후 비교에서는 개선된 기능의 대가도 확인했다. GPT-4o mini 의 응답 반환과 문서 검색은 개선됐지만 생성 토큰은 늘었고, Ollama 에서는 같은 흐름을 적용해도 검증 실패와 시간 초과가 남았다. 기능을 추가한 뒤에는 동작 여부뿐 아니라 사용할 모델과 실행 환경까지 함께 확인해야 했다.

1. 조치 단계의 누락을 확인하는 검사

현재 검증은 선택한 조치가 원문과 같은지 확인한다. 다음에는 전체 절차를 요청한 경우 필요한 단계가 빠지지 않았는지, 선행 단계가 필요한 조치인지도 검사하고 싶다. 검증 기준은 문서에 명시된 단계와 조건으로 정하고, 누락 사례와 불필요한 단계 추가 사례를 함께 테스트할 계획이다.

2. 새로운 질문과 문서로 검색 평가 확대

다음 평가에서는 구현 수정에 사용하지 않은 질문을 먼저 확정하고 문서 수도 늘리고자 한다. 특히 지금의 적용 대상 필터가 놓칠 수 있는 새로운 알림 표현을 포함해 문서 재현율과 불필요한 문서 반환을 함께 확인할 필요가 있다. 새 평가 결과를 보고 코드를 수정했다면, 그 질문은 이후 회귀 테스트로 구분하고 별도의 평가 질문을 다시 준비할 계획이다.

3. 로컬 모델의 처리 시간과 화면 상태 개선

Ollama 는 작성 모델을 8B 로 바꿨을 때 반환된 보고서의 검증 실패는 없었지만 제한 시간 초과가 남았다. 프롬프트 길이와 모델 구성을 조정할 때는 동일 질문에서 답변 품질, 처리 시간, 사용량을 함께 비교하고자 한다. 화면에서는 대상 확인이 끝난 이전 요청을 접어 최신 결과와 구분하고, 동시에 여러 요청이 들어올 때의 동작도 추가로 확인하고 싶다.

7. 실행 환경과 검증 방법

재현 방법

Windows 와 Python 3.11.9 기준이다. 동봉된 submission_v5.zip 을 풀고 해당 폴더에서 가상환경을 만든 뒤 requirements-lock-windows.txt 를 설치한다. .env.example 을 .env 로 복사하고 개인 OpenAI 키를 로컬에서 설정한다. 외부 환경변수는 .env 보다 우선한다.

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-lock-windows.txt
Copy-Item .env.example .env
powershell -ExecutionPolicy Bypass -File .\run.ps1 -Backend openai -Port 8109
```

브라우저에서 http://127.0.0.1:8109/에 접속하고 /health 에서 v5 와 모델을 확인한다. 기존 서버가 사용하는 포트는 피한다. Vue CDN 과 OpenAI 모델 및 임베딩 호출에 인터넷 연결이 필요하다. 새 PC 에서의 전체 재설치 검증은 아직 완료하지 않았다.

자동 테스트는 가상환경의 Python 으로 python -B -X utf8 -m unittest discover -s tests -p "test_*.py" -v 를 실행한다. 실제 모델 평가는 별도 기록이며 이 명령은 외부 모델 없는 48 개 회귀 테스트를 수행한다.

검증을 나눈 이유

자동 테스트에서는 모델과 도구의 응답을 고정해 제어 흐름과 예외 처리를 확인한다. 같은 입력에서 결과가 달라지면 원인을 코드에서 추적하기 쉽기 때문이다. 반면 실제 모델의 출력 형식과 MCP 연결, 화면의 대상 선택 동작은 테스트 대역만으로 확인할 수 없어 별도 브라우저 검증을 수행했다.

30 문항 비교는 기능 개선 전후의 차이를 확인하는 실험이고, 48 개 테스트는 구현이 의도한 조건을 지키는지 확인하는 검사다. 실모델 브라우저 검증은 사용자가 화면에서 질문하고 결과를 확인하는 흐름을 점검한다.

검증 자료의 기준

실험은 2026 년 10 월 20 일과 21 일의 최종 v5 결과를, 화면 검증은 10 월 20 일의 v5 결과를 기준으로 한다. 원본과 중간 버전의 기록은 비교와 오류 재현을 위해 보관했다. 아래 부록은 최종 제출 코드의 핵심 함수와 데이터 흐름을 설명한다.

부록 A. 전체 처리 구조와 데이터 흐름

최종 코드의 중심은 agent/graph.py 다. FastAPI 가 요청을 받고 ChatService 가 대화 상태를 관리하며, LangGraph 가 조사와 답변 작성 순서를 결정한다. MCP 는 관측 조회와 런북 검색에 필요한 도구를 제공한다. 각 역할은 의존성 객체를 통해 연결한다.

요청을 받는 계층

server.py 는 HTTP 요청과 서비스 시작 및 종료를 담당한다. 요청에는 사용자의 질문뿐 아니라 thread_id 와 request_id 가 포함된다. thread_id 는 대화를 구분하고, request_id 는 같은 요청의 재전송을 구분한다. 대상 확인에 응답할 때는 선택한 시스템과 clarification_id 도 전달한다.

agent/service.py 의 ChatService.start()는 요청 내용을 지문으로 만들어 저장한다. 같은 요청 ID 와 같은 내용이면 기존 실행 정보를 반환하고, 같은 ID 에 다른 내용을 보내면 409 오류를 반환한다. 같은 대화에서 이미 실행 중인 요청이 있을 때도 중복 실행을 막는다. 캐시는 프로세스 메모리에 있어 서버 재시작 후에는 유지되지 않는다.

그래프가 처리 순서를 결정하는 방식

build_graph()는 route, chat, plan, monitor, retrieve, write, finish 노드를 등록한다. route 에서 일반 대화는 chat 으로, 장애 질문은 plan 으로 보낸다. 조사 대상이 모호하면 확인 질문을 반환하고 그래프 실행을 종료한다. 대상이 정해진 장애 질문은 plan → monitor → retrieve → write → finish 순서로 처리한다.

write 에서는 보고서를 작성한 직후 검증까지 수행한다. 검증에 실패하고 작성 횟수가 3 회 미만이면 write 로 돌아간다. 따라서 검증은 별도 그래프 노드가 아니라 작성 노드 내부의 함수 호출이다. 이 구분은 코드의 실제 구조를 이해할 때 중요하다.

AgentState 에 저장하는 정보

필드	역할
query, targets	원 질문과 최종 조사 대상
observations, facts	조회에 성공한 관측과 그 관측에서 만든 사실
evidence	검색한 런북 원문과 대상, 문서 ID, 검색 점수
task_results, limitations	작업별 성공 여부와 답변에서 밝혀야 할 제한
report, validation_history	구조화된 보고서와 작성 시도별 검증 기록

상태를 나누면 조회 실패와 답변 검증 실패를 구별할 수 있다. 예를 들어 한 서비스 조회가 실패해도 다른 서비스의 observations 와 facts 는 남고, 실패 사실은 limitations 에 기록된다.

부록 B. 조사 대상 확인과 부분 실패 처리

등록된 대상과 모호한 질문을 구분

agent/targets.py 의 resolve()는 data/monitoring.json 의 등록된 시스템 이름을 읽고 질문에 포함된 이름을 찾는다. 단어 경계를 함께 검사해 비슷한 이름을 잘못 매칭하지 않도록 한다. 서비스 이름처럼 보이지만 등록되지 않은 문자열은 unresolved_names 로 분리한다.

등록된 대상이 있고 미등록 이름이 없을 때만 resolved 로 처리한다. 대상이 없거나 미등록 이름이 섞이면 needs_clarification 을 반환한다. 이 경우 임의의 기본 서비스를 조사하지 않고 화면에 선택 항목을 보낸다. v5 에서는 실제 달력에 존재하는 YYYY-MM-DD 형식 날짜를 이름 후보에서 제외해 날짜가 서비스 이름으로 잡히는 오류를 수정했다.

대상 선택 후 원 질문을 이어서 처리

route()는 확인 요청에 고유 ID 와 원 질문을 붙인다. ChatService 는 이를 저장하고, 사용자가 선택을 제출하면 consume_pending()으로 원 질문을 복원한다. 선택된 대상이 등록 목록 안에 있는지 확인한 뒤 조사로 진행한다. 이미 사용했거나 만료된 확인 요청은 거부한다.

작업 단위로 오류를 격리

agent/nodes/investigation.py 의 run_task()는 모니터링 조회와 검색을 공통 방식으로 감싼다. Semaphore 로 동시 작업 수를 제한하고, asyncio.wait_for()로 각 시도의 제한 시간을 적용한다. 함수의 기본 인자는 15 초지만 실제 그래프는 SupervisorDeps 설정을 전달한다. 기본 실행 설정은 모니터링 15 초, 런북 검색 45 초이며 각각 최대 3 회 호출한다. 재시도 전에는 0.3 초, 0.9 초를 기다리므로 각 제한 시간은 작업 전체가 아니라 한 번의 시도에 적용된다.

시간 초과와 연결 계열 오류는 재시도한다. 대상 불일치, 알 수 없는 관측 상태, 잘못된 검색 데이터처럼 재호출로 해결하기 어려운 오류는 즉시 실패로 남긴다. 반환값에는 대상, 작업 종류, 상태, 오류 코드, 재시도 수, 경과 시간이 포함된다.

두 대상을 조회할 때의 예

payments-service 와 cache-service 를 함께 조회하면 monitor()가 두 작업을 asyncio.gather()로 모은다. payments 만 성공했다면 성공 결과로 사실 목록을 만들고, cache 의 상태와 원인은 판단할 수 없다는 제한을 추가한다. 실패한 대상의 값을 추정해서 채우지는 않는다. 검색도 성공한 관측을 기준으로 수행한다.

finish()는 관측이 전혀 없으면 failed, 보고서 검증을 통과하지 못하면 validation_failed 로 구분한다. 관측과 검증이 확보된 경우에는 제한 사항을 기준으로 partial 또는 completed 를 반환한다. partial 은 실행이 중단된 예외와 다르게, 확보한 정보를 전달한 결과다.

부록 C. 런북 검색의 구현

문서와 벡터를 준비하는 과정

retrieval/index.py 의 RunbookIndex.build()는 런북을 읽고 임베딩과 함께 로컬 인덱스에 저장한다. 임베딩 모델은 text-embedding-3-small 이며 차원은 512 다. 런북이 짧고 조치 순서가 중요하기 때문에 문서를 여러 조각으로 나누지 않고 전체를 한 청크로 취급한다. 문서 ID, incident_type, 원문, 해시 등의 메타데이터를 함께 둔다.

load()는 원문 해시, 모델 정보와 벡터 차원을 검사한다. 원문이 바뀌었는데 오래된 벡터를 그대로 사용하는 상황은 INDEX_MISMATCH 오류로 처리한다. 별도 벡터 DB 서버를 운영하는 방식은 아니며 소규모 실습 문서를 대상으로 한 로컬 검색 구조다.

현재 관측에 적용 가능한 문서만 검색

search()는 상태가 healthy 이고 알림이 없으면 no_match 를 반환한다. 그 외에는 retrieval/applicability.py 의 규칙으로 실제 알림에 맞는 런북만 후보로 남긴다. 이 단계는 관련 없는 문서가 질문 표현만으로 높은 순위에 오르는 일을 줄이지만, 규칙에 없는 새 알림은 놓칠 수 있다.

남은 후보에 대해 관측 알림과 사용자 질문을 각각 임베딩한다. 정규화한 벡터의 내적으로 코사인 유사도를 계산하고 관측 점수 80%, 질문 점수 20%를 합산한다. 관측에 더 큰 비중을 둬 질문의 추측보다 조회 결과를 우선한다.

검색 점수 = 관측 알림 유사도 × 0.8 + 질문 유사도 × 0.2

점수가 0.31 이상이고 최고 점수와의 차이가 0.03 이내인 문서를 최대 3 개 반환한다. 임계값을 넘는 문서가 없으면 no_match 다.

검색 결과를 답변과 연결

검색 결과에는 system_name 과 evidence_id 를 붙인다. 같은 런북을 여러 시스템에 적용하더라도 대상별로 근거를 구분할 수 있다. 이후 보고서의 가설과 조치는 이 ID 를 사용하고 검증기는 해당 대상의 근거가 맞는지 확인한다.

그래프의 retrieve()는 검색 결과가 없으면서 활성 알림이 있는 경우, 질문 대신 알림 문자열로 검색을 보완한다. 보완 검색은 한 번의 search 작업 호출 안에서 한 차례 수행한다. 전송 오류로 작업 자체가 재시도되면 이 흐름도 다시 실행될 수 있다. 끝내 문서가 없으면 관측 사실과 추가 확인 필요성을 안내하고 문서 없는 구체 조치는 만들지 않는다.

현재 구조를 선택한 이유

문서 수를 늘리기보다 기존 6 개 문서에서 대상과 적용 조건이 맞는지를 먼저 확인했다. 그 결과 전후 비교에서 문서 추가 효과를 제외할 수 있었다. 다만 문서 수가 커지면 후보 규칙과 인덱스 갱신, 검색 품질을 다시 평가해야 한다.

부록 D. 사실과 조치 원문의 연결 및 검증

사실을 모델이 다시 쓰지 않도록 구성

agent/reporting.py 의 facts_from()은 관측에서 상태, CPU, 메모리와 알림을 읽어 Fact 객체로 만든다. 각 사실에는 관측 ID, 시스템 이름, 지표명, 값과 단위를 넣는다. 알림이 없으면 "없음"이라는 사실을 추가한다. 수치가 들어가는 사실 목록을 서버에서 구성하므로 모델이 숫자를 요약하다 바꾸는 경로를 줄인다.

모델은 조치 문장 대신 단계 ID 를 선택

agent/action_catalog.py 의 catalog()는 검색 문서의 번호가 붙은 절차를 단계별로 분리한다. 다음 번호가 나오기 전까지 이어지는 문장도 같은 단계에 포함한다. 각 항목은 source_step_id, 대상, 원문 문장, 근거 ID 를 가진다. 중복된 단계 ID 는 오류로 처리한다.

materialize()는 모델이 선택한 ID 가 목록에 있는지 검사하고, 같은 단계의 중복 선택을 거부한다. 선택 결과는 원문 순서로 정렬한다. 따라서 "미회복 시"와 "정상화 후" 같은 조건은 모델이 새로 만든 문장에 의존하지 않고 해당 원문 단계와 함께 전달된다.

가설도 확정 진단으로 표현하지 않는다. assemble()은 선택한 런북 제목을 바탕으로 점검 가설 문장을 만들고 추가 확인이 필요하다고 표시한다.

검증기가 검사하는 항목

검사	실패하는 예
대상과 사실 일치	조회하지 않은 서비스가 포함되거나 관측값이 누락됨
조치 원문 일치	단계 ID 는 맞지만 문장, 조건, 대상 또는 출처가 바뀜
가설의 근거 연결	다른 시스템의 사실과 문서를 하나의 가설에 연결함
근거와 제한 사항	사용한 문서가 인용 목록에 없거나 필수 제한이 빠짐
조치 최소 포함	문서가 검색된 대상에 대한 조치가 하나도 없음

validation.py 는 문서가 검색된 각 대상에 근거 있는 조치가 한 개 이상 포함됐는지 확인한다. 조치의 대상, 원문 단계와 출처도 함께 대조한다.

검증 실패 시의 동작

write()는 오류 코드와 필드 위치, 이전 초안을 다음 작성 시도에 전달한다. 최초 작성을 포함해 최대 3 회 검증하며 끝까지 실패하면 finish()가 가설과 조치를 비우고 확인된 사실과 검증 실패 안내만 반환한다. 검증에 실패한 초안을 정상 답변처럼 노출하지 않도록 한 것이다.

부록 E. 실행 관리와 테스트 구성

의존성 주입과 테스트 대역

agent/deps.py 의 SupervisorDeps 는 모델, MCP 클라이언트와 실행 설정을 묶는다. build_graph()는 이 객체를 각 노드에 전달한다. 테스트에서는 실제 모델 대신 고정 응답 모델을, 실제 MCP 대신 FixtureMCP 를 넣을 수 있다. 그래프 코드를 바꾸지 않고 성공, 부분 실패, 잘못된 조치 ID 와 재작성 상한을 반복 검증하는 데 사용했다.

이벤트 루프를 막지 않는 모델 호출

OpenAI 어댑터의 동기 HTTP 호출은 asyncio.to_thread()로 실행한다. 응답을 기다리는 동안 이벤트 루프가 다른 요청을 처리할 수 있게 하기 위해서다. 동기 HTTP 요청이 완료되면 결과를 받아 원래 처리 흐름을 이어간다.

시작 실패 시 자원 정리

server.py 의 lifespan()은 AsyncExitStack 으로 종료 절차를 관리한다. MCP 종료 콜백을 등록한 뒤 시작을 시도하고 SQLite 저장소의 close 도 등록한다. 초기화 중 예외가 나거나 정상 종료할 때 등록된 정리 작업이 역순으로 실행된다. MCP 클라이언트 내부에서도 부분적으로 시작된 연결을 정리해 실패한 초기화가 자원을 남기지 않도록 했다.

진행 상태와 화면

ChatService.run()은 그래프의 updates 이벤트를 받아 진행 기록을 누적한다. server.py 의 스트림 경로는 이 기록과 최종 응답을 SSE 로 전달한다. 화면은 대상 선택, 작업 결과, 검증 상태와 런북 내용을 보여 준다. 노드 완료 시점의 갱신이며 모델의 토큰이 생성되는 즉시 한 글자씩 표시하는 스트리밍과는 다르다.

테스트로 확인한 경계

테스트 파일	확인하는 내용
test_features.py	대상 확인, 복합 대상, 부분 실패와 기본 보고서 흐름
test_api_retrieval.py	API 응답과 검색의 정상 및 오류 경로
test_action_binding.py	조치 단계의 원문 연결과 잘못된 선택 처리
test_repair.py	검증 오류 전달과 재작성 종료 조건
test_runtime_fixes.py	날짜 인식, 비동기 호출, 초기화 실패 자원 정리

기존 34 개와 신규 14 개를 합친 48 개 테스트가 통과했다. 모델 출력과 실제 연결을 포함한 검증은 본문의 30 문항 비교와 브라우저 결과에서 별도로 확인했다.

부록 F. 실제 실행 화면 1 대상 확인

질문에 서비스 이름이 없어 조사 전에 대상 선택을 요청한 화면이다. 이 상태에서는 모니터링 도구를 호출하지 않았다. 아래는 최종 v5 와 실제 OpenAI 모델을 사용한 검증 화면이다.



그림 1 조사 대상 선택 전. 서비스 목록에서 대상을 지정해야 조사를 진행한다.

부록 F. 실제 실행 화면 2 선택 후 결과

auth-service 를 선택한 뒤 원 질문을 이어서 처리한 결과다. 해당 대상의 CPU, 메모리와 알림을 사실로 표시하고, 원인 후보는 추가 확인이 필요한 가설로 구분한다.



그림 2 조사 대상 선택 후. 상단에 대상 확정과 조회, 작성 및 검증 상태가 표시된다.

부록 F. 실제 실행 화면 3 조치 조건 보존

전체 1~5 단계를 요청한 보충 질문의 결과다. 4 단계의 미회복 시 조건과 5 단계의 정상화 후 조건을 서로 다른 원문 단계로 확인할 수 있다. 실제 조치는 실행하지 않았다.

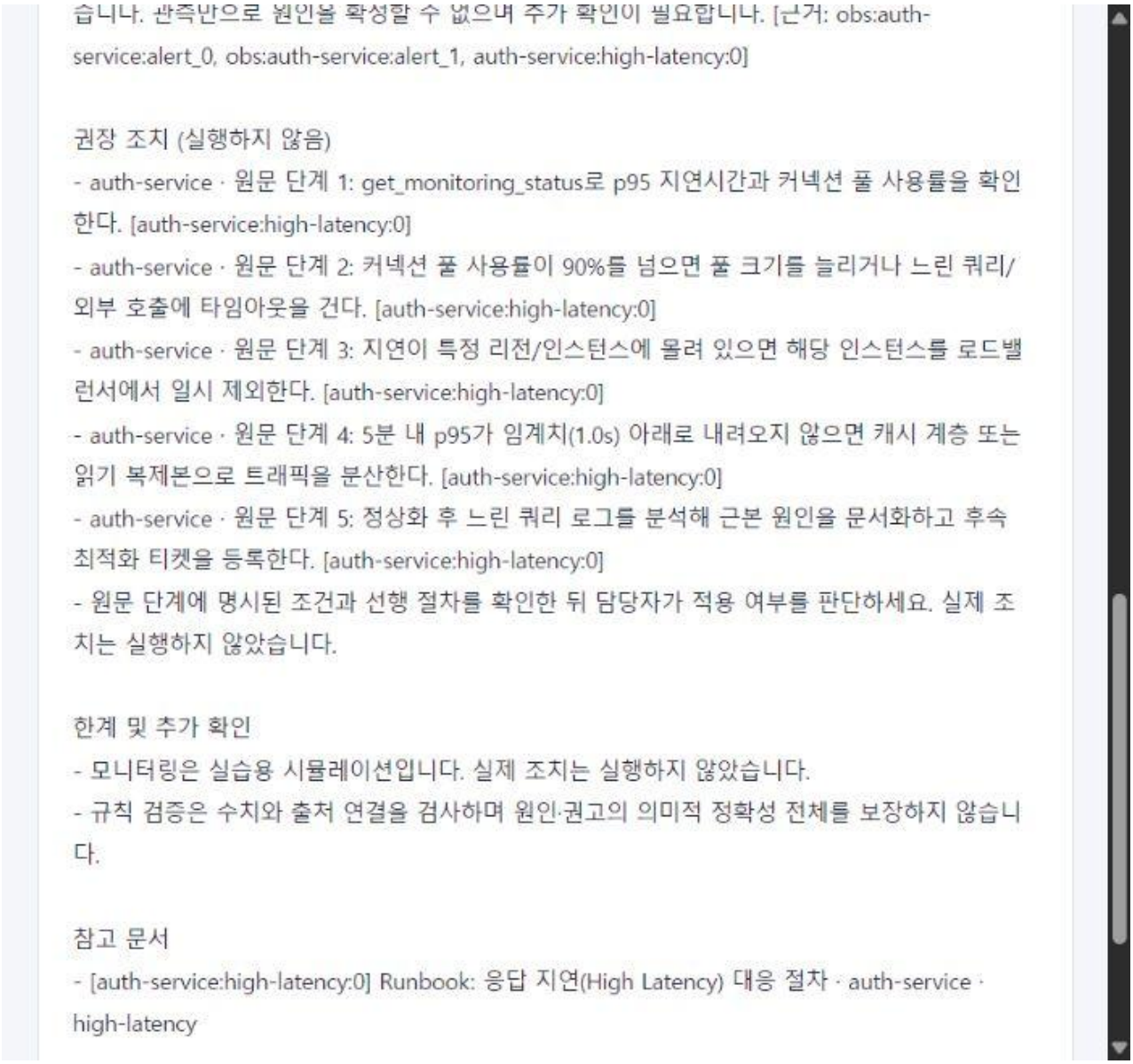


그림 3 전체 절차와 조건을 표시한 응답. 최초 질문에서 2 단계만 선택된 결과와 구분한다.

부록 G. 개발 과정의 Ollama 측정 기록

아래는 Qwen3 0.6B 와 8B 를 사용한 원본 및 중간 버전의 측정 기록이다. 모델 구성은 분류 / 보고서 작성 모델 순서다. 각 구성은 같은 30 문항, 문항당 240 초 제한을 적용한 단회 실행이다. 최종 v5 의 동일 환경 비교는 본문 3.4 절에 정리했다.

버전	모델 구성	저장 건수	응답 반환	검증 실패	시간 초과	기타 오류
원본	0.6B / 0.6B	30/30	28	미검사	0	2
원본	0.6B / 8B	30/30	24	미검사	5	1
원본	8B / 8B	30/30	10	미검사	19	1
v1	0.6B / 0.6B	30/30	29	12	1	0
v1	0.6B / 8B	30/30	25	0	5	0
v1	8B / 8B	30/30	12	0	18	0
v2	0.6B / 0.6B	30/30	30	15	0	0
v2	0.6B / 8B	30/30	19	0	11	0
v2	8B / 8B	17/30	0	0	2	15

검증 실패는 응답 반환 건수에 포함되며 정상 분석 완료와 구분했다. 시간 초과는 문항 제한 초과와 도구 시간 초과로 실행이 종료된 건수를 합산했다. 기타 오류에는 테스트에 주입한 영구 도구 오류도 포함한다. 원본은 개선 후와 같은 근거 검증기가 없어 검증 실패 수를 직접 비교하지 않았다.

v2 의 8B / 8B 구성은 17 건만 저장되어 미완료다. 기타 오류 15 건은 작업 실행 오류로 기록되었고, 저장된 응답만으로 세부 원인을 확정하지 않았다. 전체 30 문항의 성능으로 환산하지 않았다. 과거 기록에는 모델 다운로드와 다른 실행의 영향도 있었으므로, 이 표는 개발 과정의 시도와 실패 유형을 보여 주는 자료다.

부록 H. 개선 전후 실측 결과 화면 1

저장된 원본과 v5 의 실제 모델 실행 기록을 조회하는 화면이다. 3.2 절의 반복 측정 수치와 같은 기록을 사용했다. 서비스 실행 화면은 부록 F 에, 이 화면에 대응하는 문항별 기록은 제출 ZIP 에 포함했다.

원시 측정 기록 조회 | 원본과 최종 v5 | 저장된 실모델 실행 결과

동일 질문의 원본과 v5 반복 측정

GPT-4o mini, 동일 30문항, 온도 0, 문항당 240초 제한
질문, 관측값, 런북 6개를 고정한 원본→v5 비교

회차	응답 반환 전→후	문서 재현율 전→후	공통 응답 중앙값 전→후
1차	27/30 → 30/30	76.5% → 100.0%	10.91초 → 8.63초
2차	27/30 → 30/30	70.6% → 100.0%	6.52초 → 6.48초
3차	27/30 → 30/30	76.5% → 100.0%	6.04초 → 6.94초

실제 API 사용량

회차	원본 생성 토큰	v5 생성 토큰	v5 임베딩 토큰	재작성 전→후
1차	26,967	63,585	936	0 → 1
2차	27,142	59,012	936	0 → 0
3차	26,909	59,289	936	0 → 0

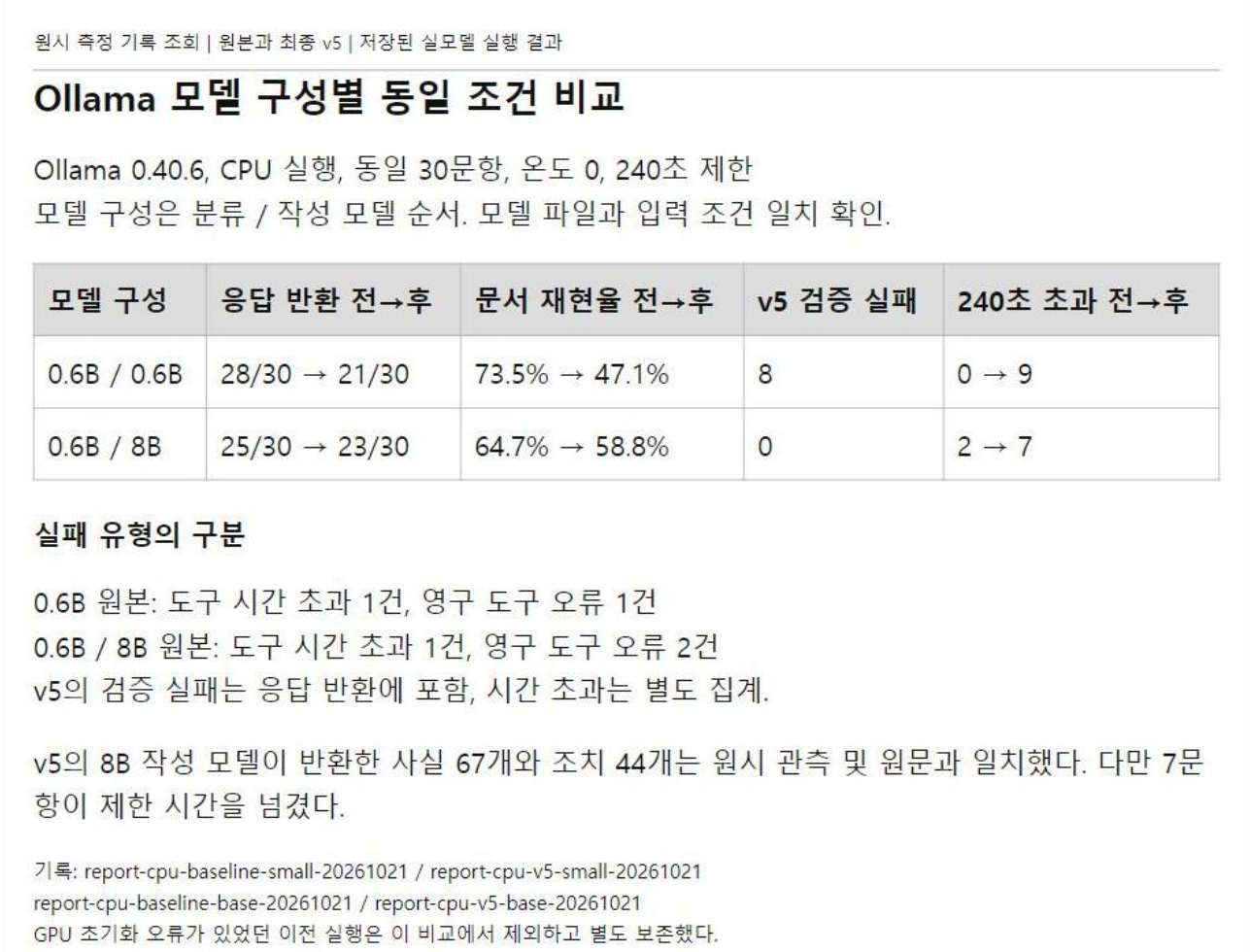
재현율: 런북이 필요한 17문항의 평균. 지연: 양쪽 모두 응답한 27문항.
생성 토큰은 입력과 출력의 합이며, 실제 청구 금액이 아니다.

기록: baseline-openai-v1 / submission-v5-direct-20261020_203210; report-baseline-mini-r2-20261021 / report-v5-mini-r2-20261021; report-baseline-mini-r3-20261021 / report-v5-mini-r3-20261021

그림 4. GPT-4o mini 의 세 회차 전후 비교, API 토큰 사용량과 재작성 횟수. 저장 기록 조회 화면의 브라우저 캡처.

부록 H. 개선 전후 실측 결과 화면 2

동일한 CPU 실행 환경에서 Qwen3 모델 구성을 바꿔 측정한 원본과 v5 의 비교 화면이다. 응답 반환과 검증 실패, 제한 시간 초과를 구분했다. 3.4 절의 모델 선택 근거와 연결된다.



원시 측정 기록 조회 | 원본과 최종 v5 | 저장된 실모델 실행 결과

Ollama 모델 구성별 동일 조건 비교

Ollama 0.40.6, CPU 실행, 동일 30문항, 온도 0, 240초 제한
모델 구성은 분류 / 작성 모델 순서. 모델 파일과 입력 조건 일치 확인.

모델 구성	응답 반환 전→후	문서 재현율 전→후	v5 검증 실패	240초 초과 전→후
0.6B / 0.6B	28/30 → 21/30	73.5% → 47.1%	8	0 → 9
0.6B / 8B	25/30 → 23/30	64.7% → 58.8%	0	2 → 7

실패 유형의 구분

0.6B 원본: 도구 시간 초과 1건, 영구 도구 오류 1건
0.6B / 8B 원본: 도구 시간 초과 1건, 영구 도구 오류 2건
v5의 검증 실패는 응답 반환에 포함, 시간 초과는 별도 집계.

v5의 8B 작성 모델이 반환한 사실 67개와 조치 44개는 원시 관측 및 원문과 일치했다. 다만 7문항이 제한 시간을 넘겼다.

기록: report-cpu-baseline-small-20261021 / report-cpu-v5-small-20261021
report-cpu-baseline-base-20261021 / report-cpu-v5-base-20261021
GPU 초기화 오류가 있었던 이전 실행은 이 비교에서 제외하고 별도 보존했다.

그림 5. Ollama 동일 조건 전후 비교. 저장 기록 조회 화면의 브라우저 캡처이며, 실패 결과도 포함했다.